Python Matplotlib



What is Matplotlib?

- Matplotlib is a low level graph plotting library in python that serves as a visualization utility.
- Matplotlib was created by John D. Hunter.
- Matplotlib is open source and we can use it freely.
- Matplotlib is mostly written in python, a few segments are written in C, Objective-C and Javascript for Platform compatibility.

Installation of Matplotlib

- If you have Python and PIP already installed on a system, then installation of Matplotlib is very easy.
- Install it using this command:

```
C:\Users\Your Name>pip install matplotlib
```

Import Matplotlib

Once Matplotlib is installed, import it in your applications by adding the import *module* statement:

```
import matplotlib
```

Checking Matplotlib Version

The version string is stored under __version__ attribute.

```
import matplotlib

print(matplotlib.__version__)
```

Matplotlib Pyplot

Pyplot

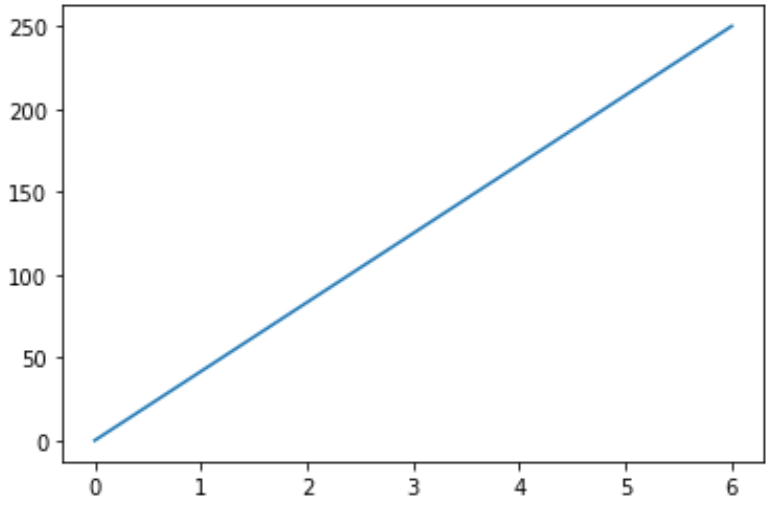
Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

import matplotlib.pyplot as plt

Now the Pyplot package can be referred to as plt.

Example

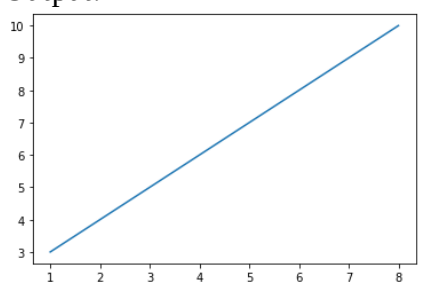
Draw a line in a diagram from position (0,0) to position (6,250):

import matplotlib.pyplot as plt import numpy as np xpoints = np.array([0, 6]) ypoints = np.array([0, 250]) plt.plot(xpoints, ypoints) plt.show()	Output: 

Plotting x and y points

- The plot() function is used to draw points (markers) in a diagram.
- By default, the plot() function draws a line from point to point.
- The function takes parameters for specifying points in the diagram.
- Parameter 1 is an array containing the points on the **x-axis**.
- Parameter 2 is an array containing the points on the **y-axis**.
- If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Draw a line in a diagram from position (1, 3) to position (8, 10):

import matplotlib.pyplot as plt import numpy as np xpoints = np.array([1, 8]) ypoints = np.array([3, 10]) plt.plot(xpoints, ypoints) plt.show()	Output: 

Plotting Without Line

To plot only the markers, you can use shortcut string notation parameter 'o', which means 'rings'.

Example

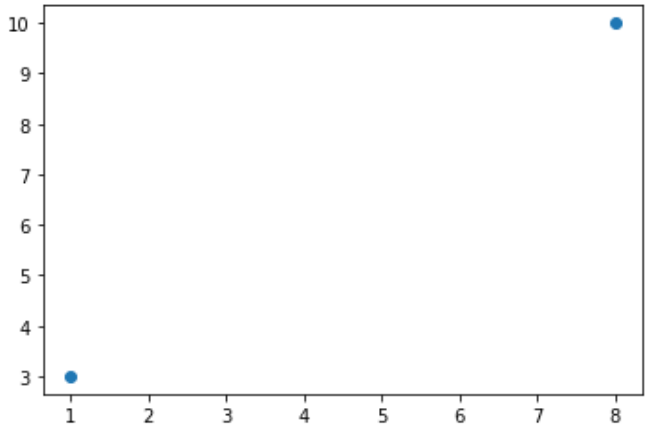
Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```

Output;



Multiple Points

You can plot as many points as you like, just make sure you have the same number of points in both axis.

Example

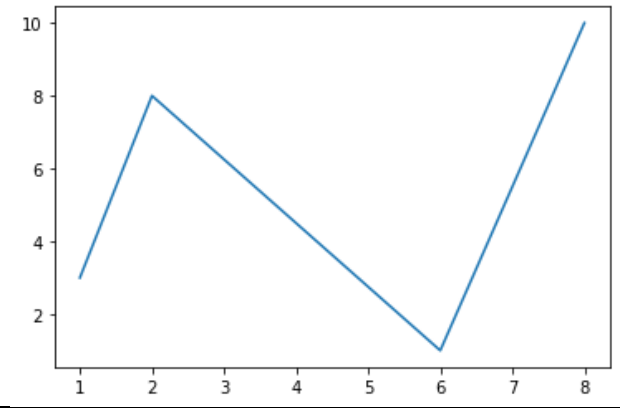
Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

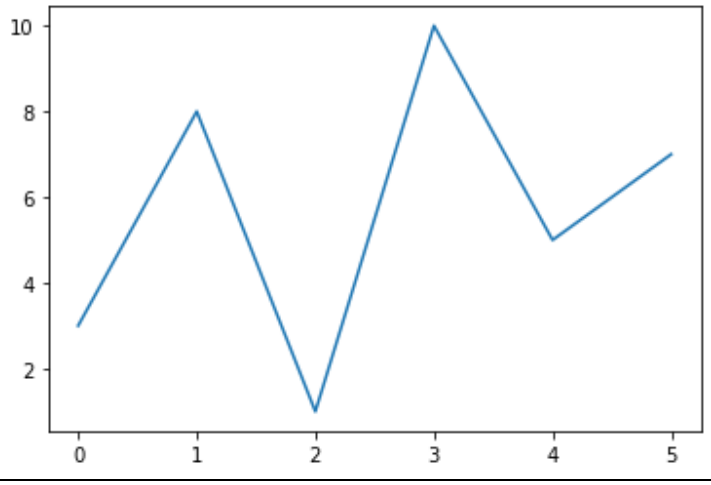
Output:



Default X-Points

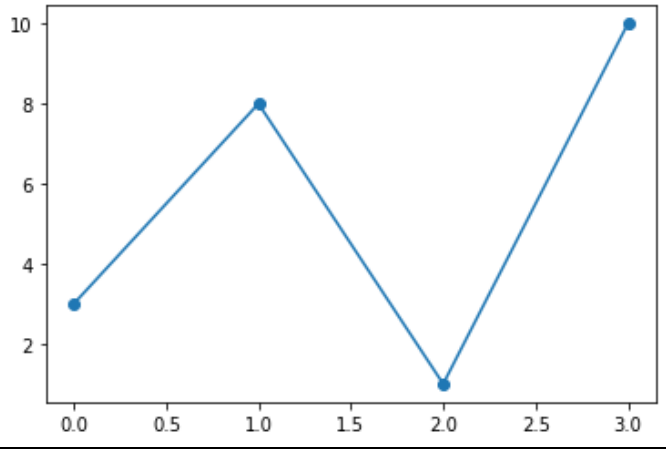
If we do not specify the points on the x-axis, they will get the default values 0, 1, 2, 3 etc., depending on the length of the y-points.

So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

import matplotlib.pyplot as plt import numpy as np ypoints = np.array([3, 8, 1, 10, 5, 7]) plt.plot(ypoints) plt.show()	Output: 

Markers

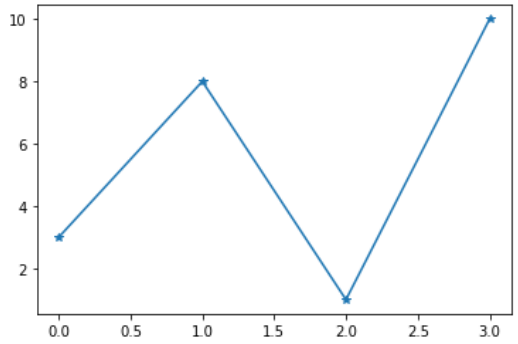
You can use the keyword argument marker to emphasize each point with a specified marker:

import matplotlib.pyplot as plt import numpy as np ypoints = np.array([3, 8, 1, 10]) plt.plot(ypoints, marker = 'o') plt.show()	Output: 

Mark each point with a star:

```
...
plt.plot(ypoints, marker = '*')
...
```

Output:



Marker Reference

You can choose any of these markers:

Marker	Description	Marker	Description
'o'	Circle	'H'	Hexagon
'*'	Star	'h'	Hexagon
'.'	Point	'v'	Triangle Down
','	Pixel	'^'	Triangle Up
'x'	X	'<'	Triangle Left
'X'	X (filled)	'>'	Triangle Right
'+'	Plus	'1'	Tri Down
'P'	Plus (filled)	'2'	Tri Up
's'	Square	'3'	Tri Left
'D'	Diamond	'4'	Tri Right
'd'	Diamond (thin)	'|'	Vline
'p'	Pentagon	'_'	Hline

Format Strings fmt

You can also use the shortcut string notation parameter to specify the marker.

This parameter is also called fmt, and is written with this syntax:

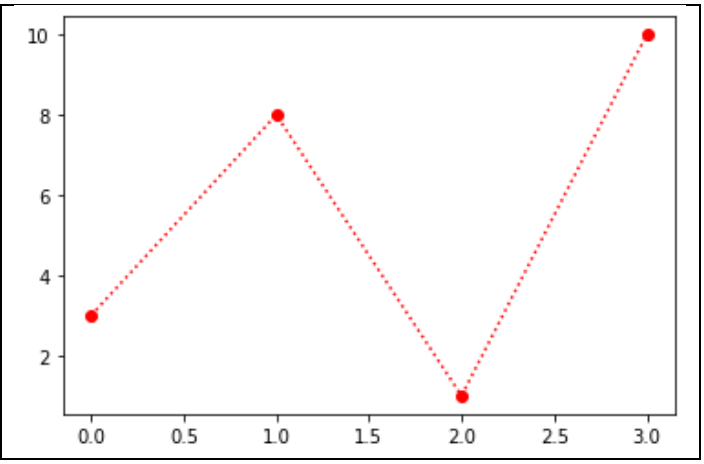
marker|*line*|*color*

Mark each point with a circle:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, 'o:r')
plt.show()
```



Line Reference

Line Syntax	Description
'-'	Solid line
':'	Dotted line
'--'	Dashed line
'-.'	Dashed/dotted line

Color Reference

Color Syntax	**Description**
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

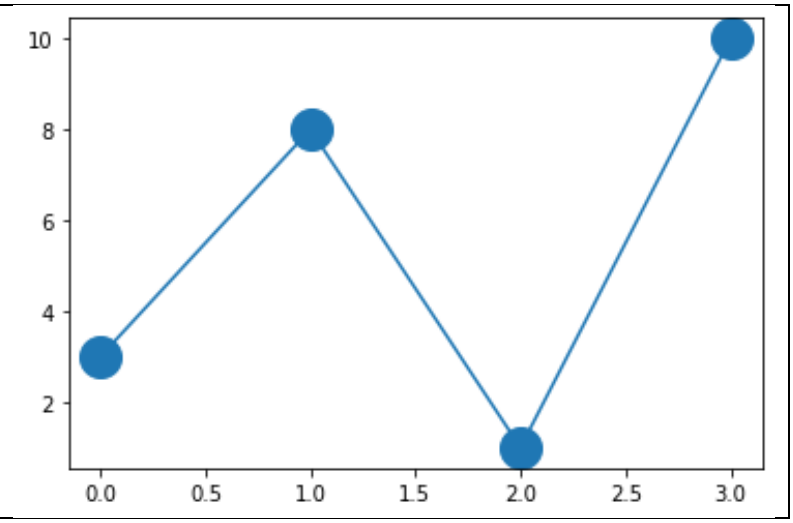
Marker Size

You can use the keyword argument markersize or the shorter version, ms to set the size of the markers:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o',
ms = 20)
plt.show()
```



Marker Color

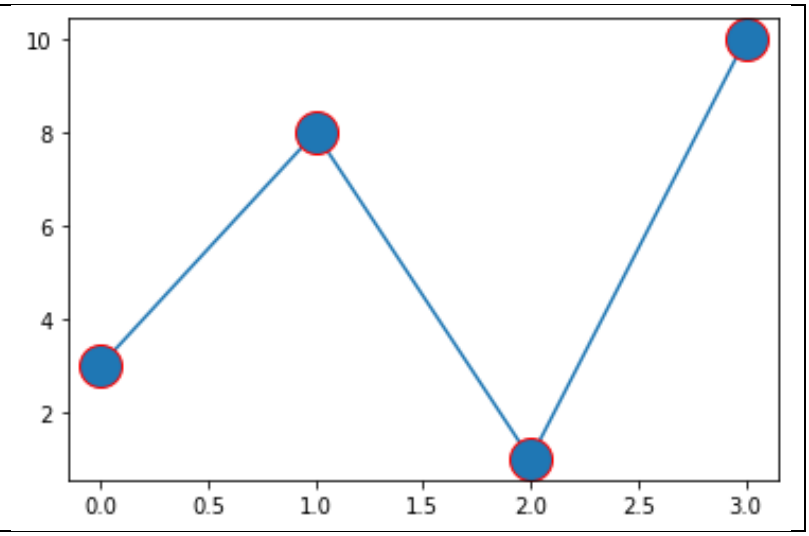
You can use the keyword argument markeredgecolor or the shorter mec to set the color of the *edge* of the markers:

Set the EDGE color to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o',
ms = 20, mec = 'r')
plt.show()
```

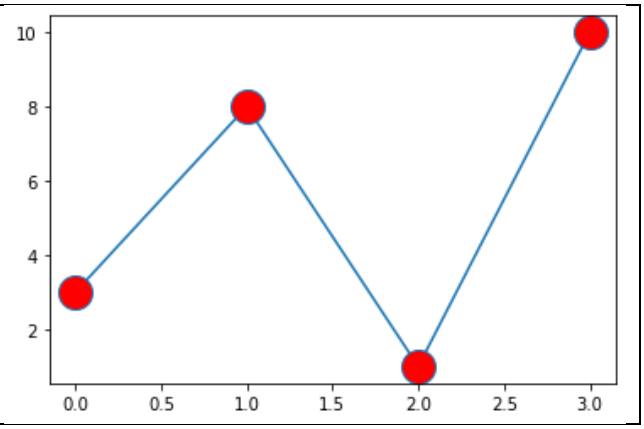


Set the FACE color to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, marker = 'o', ms = 20, mfc
= 'r')
plt.show()
```



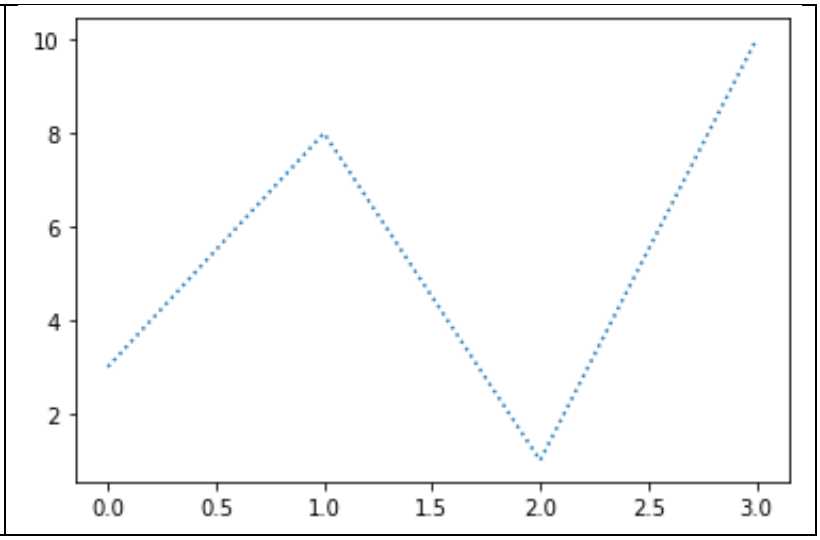
Linestyle

You can use the keyword argument linestyle, or shorter ls, to change the style of the plotted line:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

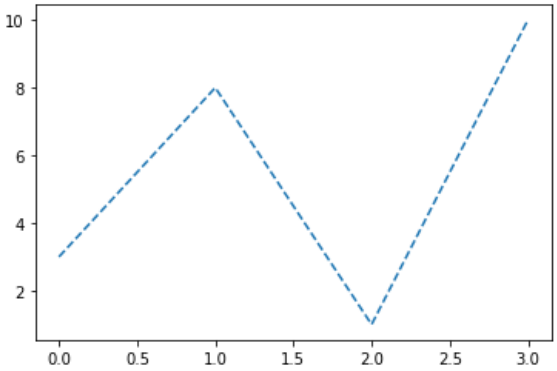
plt.plot(ypoints, linestyle =
'dotted')
plt.show()
```



Use a dashed line:

```
plt.plot(ypoints, linestyle = 'dashed')
```

Output:



Shorter Syntax

The line style can be written in a shorter syntax:

linestyle can be written as ls.

dotted can be written as :.

dashed can be written as --.

Line Color

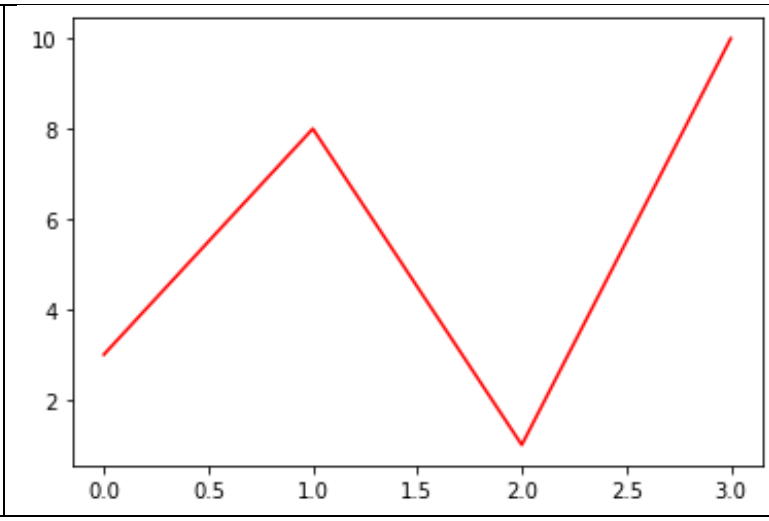
You can use the keyword argument color or the shorter c to set the color of the line:

Set the line color to red:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, color = 'r')
plt.show()
```



Line Width

You can use the keyword argument linewidth or the shorter lw to change the width of the line.

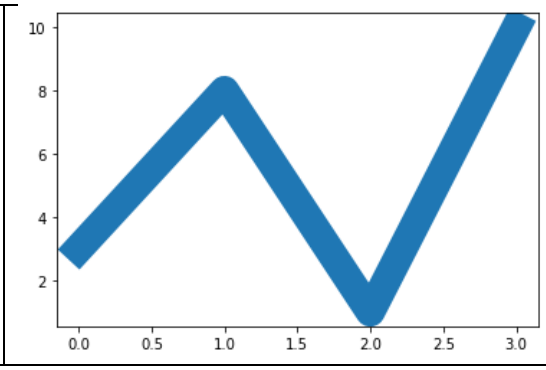
The value is a floating number, in points:

Plot with a 20.5pt wide line:

```
import matplotlib.pyplot as plt
import numpy as np

ypoints = np.array([3, 8, 1, 10])

plt.plot(ypoints, linewidth = '20.5')
plt.show()
```



Multiple Lines

You can plot as many lines as you like by simply adding more plt.plot() functions:

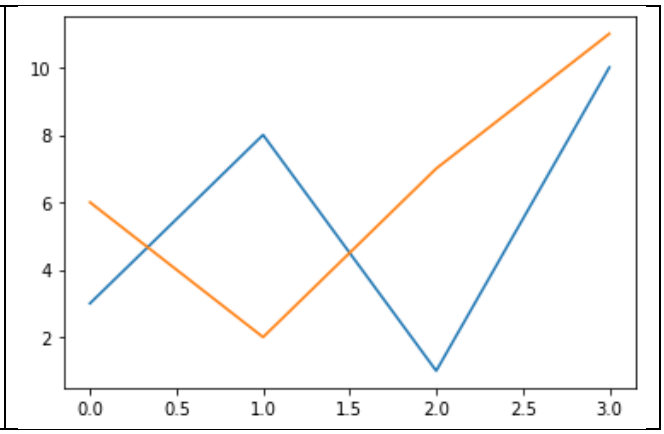
Draw two lines by specifying a `plt.plot()` function for each line:

```
import matplotlib.pyplot as plt
import numpy as np

y1 = np.array([3, 8, 1, 10])
y2 = np.array([6, 2, 7, 11])

plt.plot(y1)
plt.plot(y2)

plt.show()
```



Matplotlib Labels and Title

Create Labels for a Plot

With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

Add labels to the x- and y-axis:

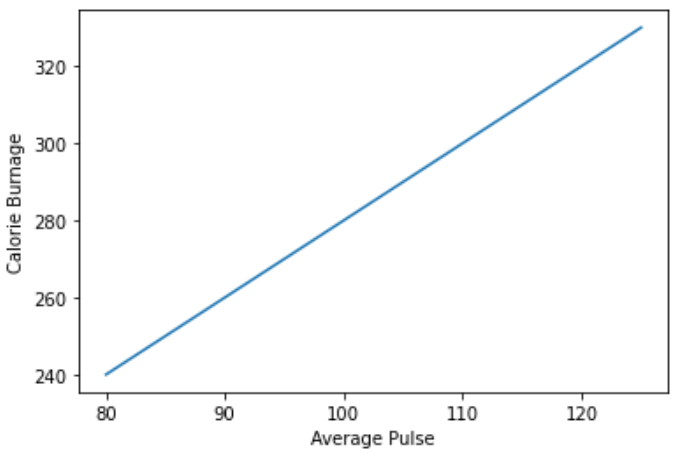
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110,
115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290,
300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```



Create a Title for a Plot

With Pyplot, you can use the title() function to set a title for the plot.

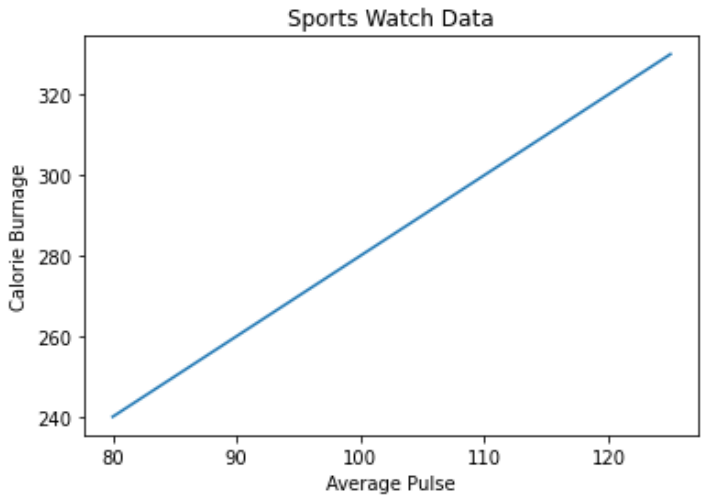
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110,
115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290,
300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```



Set Font Properties for Title and Labels

You can use the fontdict parameter in xlabel(), ylabel(), and title() to set font properties for the title and labels.

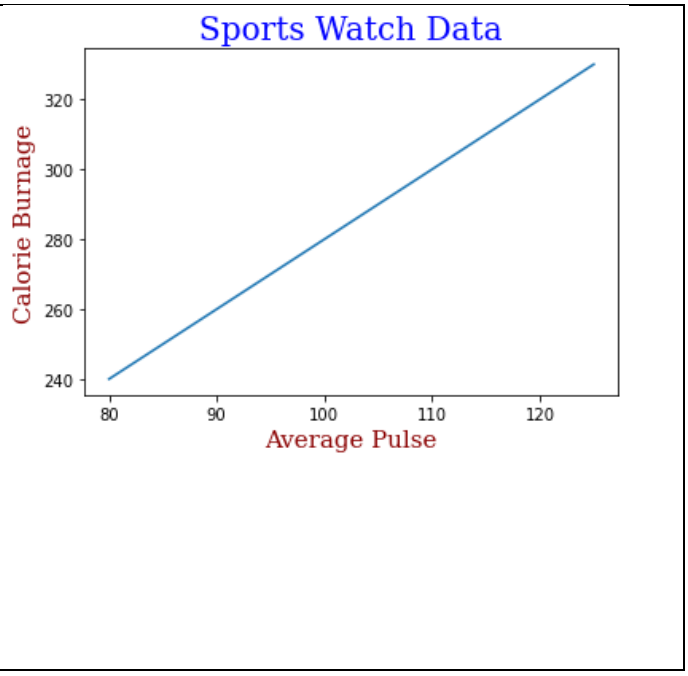
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110,
115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290,
300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 =
{'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x, y)
plt.show()
```



Position the Title

You can use the loc parameter in title() to position the title.

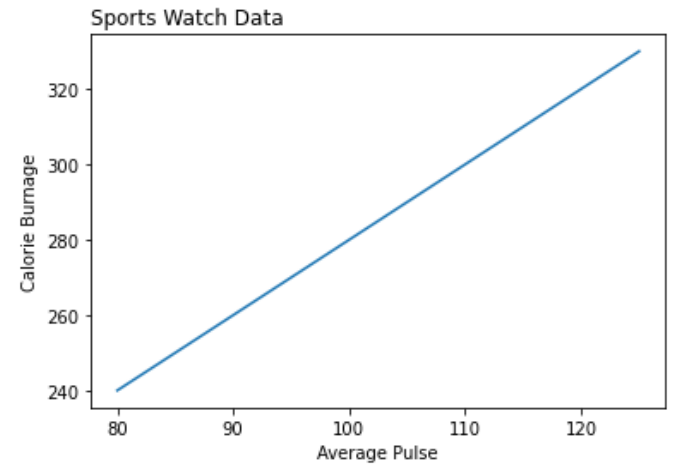
Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
plt.show()
```



Add grid lines to the plot:

```
import numpy as np
import matplotlib.pyplot as plt

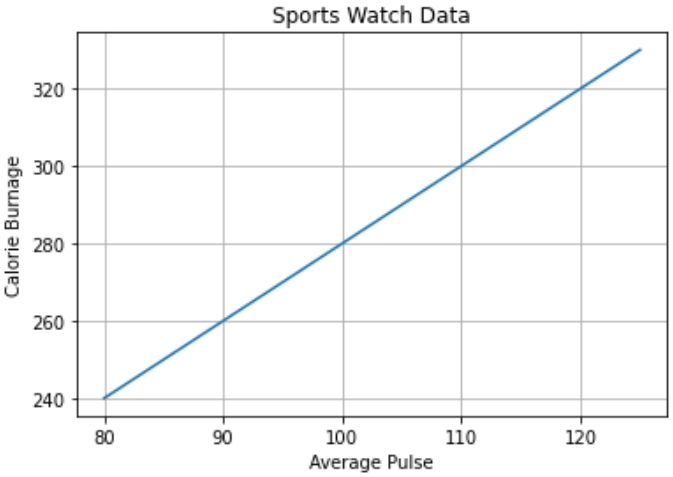
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid()

plt.show()
```



Display only grid lines for the x-axis:

```
import numpy as np
import matplotlib.pyplot as plt

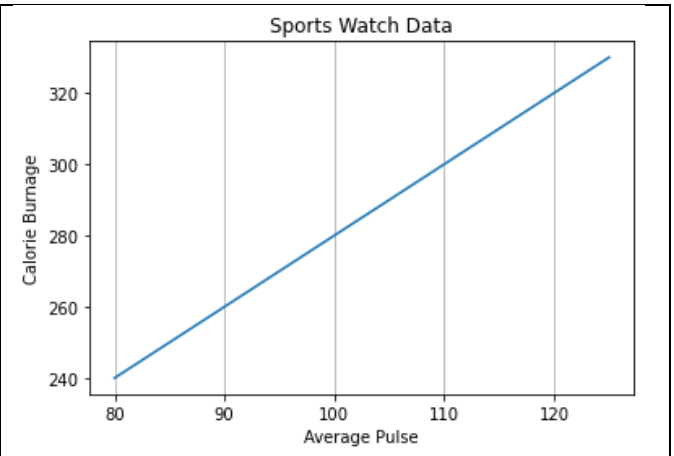
x =
np.array([80, 85, 90, 95, 100, 105,
110, 115, 120, 125])
y =
np.array([240, 250, 260, 270, 280, 2
90, 300, 310, 320, 330])

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)

plt.grid(axis = 'x')

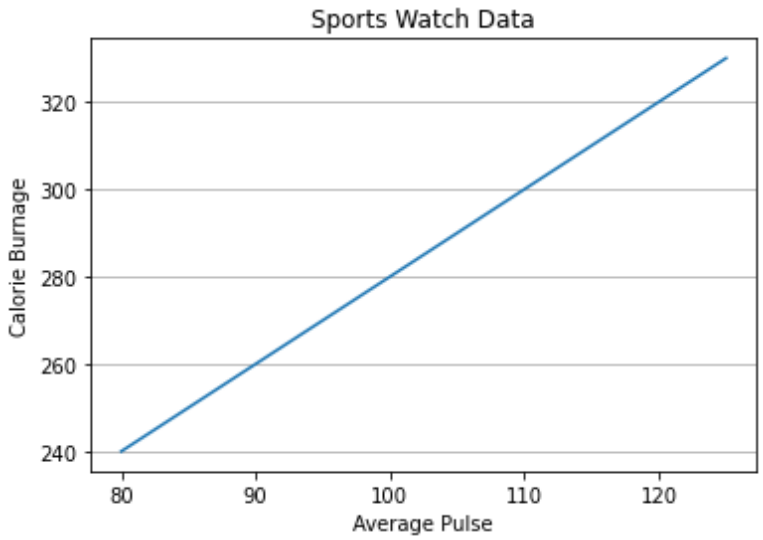
plt.show()
```



Display only grid lines for the y-axis:

```
plt.grid(axis = 'y')
```

Output:



Display Multiple Plots

With the subplot() function you can draw multiple plots in one figure:

Draw 2 plots:

```
import matplotlib.pyplot as plt
import numpy as np

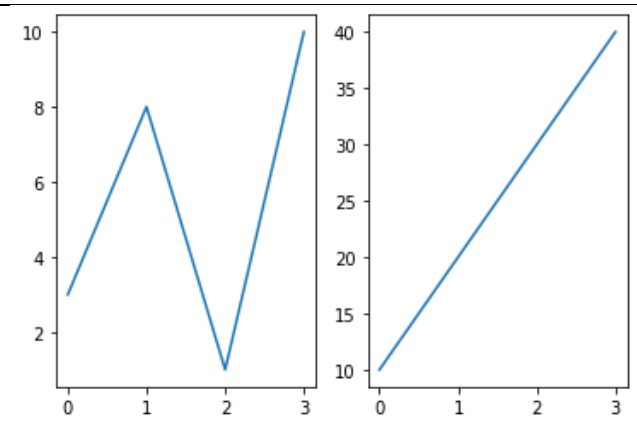
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```



Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])

plt.pie(y)
plt.show()
```



Labels

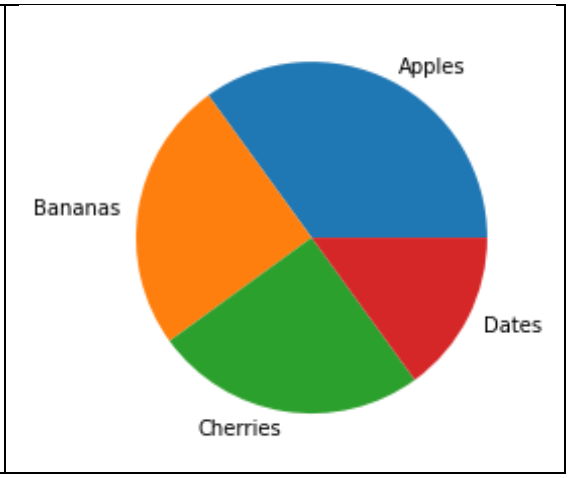
Add labels to the pie chart with the label parameter.

The label parameter must be an array with one label for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels =
["Apples", "Bananas", "Cherries", "Dates"
]

plt.pie(y, labels = mylabels)
plt.show()
```



Explode

Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge.

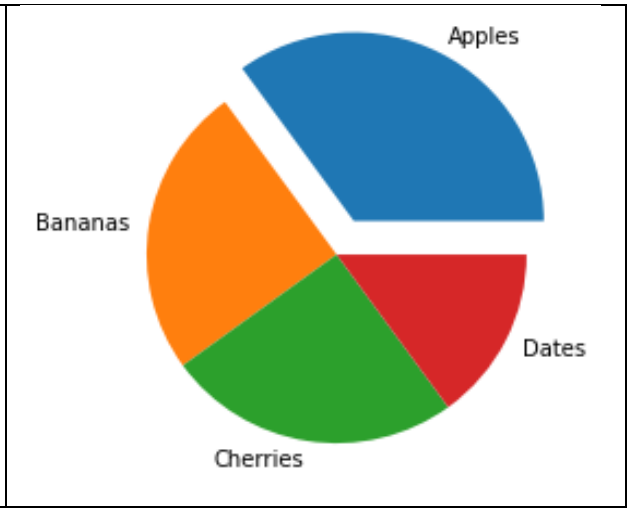
Each value represents how far from the center each wedge is displayed:

Pull the "Apples" wedge 0.2 from the center of the pie:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels =
["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode =
myexplode)
plt.show()
```



Add a legend:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas",
"Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend()
plt.show()
```

